SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	K.Kavyanjali	**Reg. No.:**	192324133
Section / Batch:	B.Tech-AI&DS	**Date:**	16/07/2026

Code of Conduct

I s.sahithya(192472186) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.Kavyanjali

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

1] Intelligent Email and Password Validator using Regular Expressions

PROBLEM UNDERSTANDING:

Develop a Python program to validate an Email Address, Password, and Mobile Number using Regular Expressions based on the specified validation rules.

ALGORITHM:

1) Import the re module.
2) Read email, password, and mobile number.
3) Validate each using regular expressions.
4) Display valid or invalid messages.

CODE:

```
import re
email=input("Enter Email: ")
password=input("Enter Password: ")
mobile=input("Enter Mobile Number: ")
if re.fullmatch(r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$',email):
    print("Valid Email")
else:
    print("Invalid Email")
if re.fullmatch(r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$',password):
    print("Strong Password")
else:
    print("Weak Password")
if re.fullmatch(r'^[6-9]\d{9}$',mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

OUTPUT:

```
Enter Email: Kavya23@gmail.com
Enter Password: 123kavya
Enter Mobile Number: 8917776362
Valid Email
Weak Password
Valid Mobile Number

=== Code Execution Successful ===
```

COMPETITIVE PERFORMANCE:

The program validates user inputs quickly using Python Regular Expressions and executes efficiently.

TEST CASE HANDLING:

- Valid Email
- Invalid Email
- Strong Password
- Weak Password
- Valid Mobile
- Invalid Mobile

RESULT:

The program successfully validates Email, Password, and Mobile Number.

2] Design a DFA Simulator

PROBLEM UNDERSTANDING:

Develop a Python program to simulate a DFA that accepts strings ending with **"ab"**.

ALGORITHM:

1) Read the input string.
2) Start from state q0.
3) Change states according to the input characters.
4) If the final state is q2, accept the string.
5) Otherwise reject it.

CODE:

```
state="q0"
string=input("Enter String: ")
for ch in string:
    if state=="q0":
        if ch=='a':
            state="q1"
    elif state=="q1":
        if ch=='a':
            state="q1"
        elif ch=='b':
            state="q2"
    elif state=="q2":
        if ch=='a':
            state="q1"
        else:
```

```
        state="q0"
if state=="q2":
   print("Accepted")
else:
   print("Rejected")
```

OUTPUT:

```
Enter String: aaaab
Accepted

=== Code Execution Successful ===
```

COMPETITIVE PERFORMANCE :

The DFA processes one character at a time, making it efficient with linear time complexity.

TEST CASE HANDLING :

Input	Output
ab	Accepted
aaab	Accepted
aba	Rejected
baba	Rejected

RESULT:

The DFA simulator correctly accepts strings ending with **"ab"** and rejects others.

3] Smart Pattern Matching Engine

PROBLEM UNDERSTANDING:

Develop a Python program to search Dates, Phone Numbers, Hashtags, Mentions, Prefixes, and Suffixes using Regular Expressions.

ALGORITHM:

1) Import the re module
2) Read the input text.
3) Display the search menu.
4) Accept the user's choice.
5) Apply the appropriate Regular Expression.
6) Display matching results.

CODE:

```
import re
text=input("Enter Text: ")
print("1.Date")
print("2.Phone")
print("3.Hashtag")
print("4.Mention")
choice=int(input("Enter Choice: "))
if choice==1:
    print(re.findall(r'\d{2}/\d{2}/\d{4}',text))
elif choice==2:
    print(re.findall(r'[6-9]\d{9}',text))
elif choice==3:
    print(re.findall(r'#\w+',text))
elif choice==4:
    print(re.findall(r'@\w+',text))
```

OUTPUT:

```
Enter Text: Meeting on 16/07/2026 Call 9121050702 #NLP @OpenAI
1.Date
2.Phone
3.Hashtag
4.Mention
Enter Choice: 2
['9121050702']

=== Code Execution Successful ===
```

COMPETITIVE PERFORMANCE:

The program efficiently searches text using Python Regular Expressions and produces results with minimal execution time.

TEST CASE HANDLING:

Choice	Input Available	Output
1	12/09/2026	Date Found
2	9876543210	Phone Found
3	#NLP	Hashtag Found
4	@OpenAI	Mention Found

RESULT:

The program successfully searches and displays matching patterns using Regular Expressions.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____